

Stochastic Green Edge (SGE)

Cómo funciona la **demo**

Una API Gateway que decide, petición por petición, cuándo vale la pena gastar energía en revisar a fondo

Panel en vivo · Inspector paso a paso · Generador de tráfico

Índice

1. **Qué es** la demo (dos formas de verla)
2. Cómo **ejecutarla** (comandos)
3. La **arquitectura** del gateway (endpoints)
4. El **panel en vivo** (dashboard)
5. El **Inspector paso a paso** (las 5 etapas)
6. Los **4 tipos de petición** de ejemplo
7. Recorrido real de cada tipo
8. El **"dado"**: la naturaleza estocástica
9. El **generador de carga** y la demo CLI
10. Cómo **leer los resultados**

1. ¿Qué es la demo?

La demo aplica la defensa adaptativa de SGE a **peticiones HTTP reales** y muestra, en vivo, la **seguridad** lograda y la **energía / CO₂ ahorrados** frente a inspeccionar el 100 % del tráfico.

Hay **dos formas** de verla:

Demo	Comando	Qué muestra
Web (principal)	<code>python demo/gateway.py</code>	Pasarela HTTP + panel en vivo + Inspector
CLI	<code>python main.py --demo-nash</code>	Cálculo del Nash + reporte en texto

Solo usa la **biblioteca estándar de Python** + `numpy` . No hay que instalar nada más.

2. Cómo ejecutarla

Demo web (desde la raíz del repo):

```
python demo/gateway.py          # abre http://localhost:8000
python demo/gateway.py --port 9000
```

Luego abre <http://localhost:8000> en el navegador.

Generar tráfico real (opcional, en otra terminal):

```
python demo/load_test.py --n 2000
```

Demo CLI (sin navegador):

```
python main.py --demo-nash --requests 2000 --seed 7
```

3. Arquitectura del gateway

Un `ThreadingHTTPServer` de la librería estándar expone rutas que ejecutan el **mismo pipeline** `sgc/` que el resto del proyecto.

Ruta	Función
<code>/</code>	Panel en vivo (dashboard)
<code>/inspector</code>	Inspector animado paso a paso
<code>/sample?kind=...</code>	Procesa 1 petición de ejemplo y actualiza métricas
<code>/simulate?n=...</code>	Procesa N peticiones con la mezcla 80/15/5
<code>/inspect?kind=...</code>	Devuelve todos los valores intermedios de una petición
<code>/roll?kind=...&n=...</code>	Repite el muestreo N veces (el "dado")
<code>/metrics</code>	Estado agregado (lo que pinta el panel)
<code>/reset</code>	Reinicia los acumuladores

Un objeto `Gateway` protegido por un `lock` mantiene el estado (`EdgeLayer` ,

4. El panel en vivo (/)

Botones: **Generar 500 / 2000** · enviar **legítimo / bot / SQLi / prompt injection** · **Reiniciar**.

Tarjetas que se refrescan cada 1.5 s:

- **Peticiones procesadas** y su mezcla (legit · bots · avanzados)
- **Mitigación global y de avanzados** (%)
- **Escalados a Serverless** (%) y **cold starts**
- **Energía ahorrada** (%) — barra adaptativo vs. tradicional
- **CO₂ evitado** (proyección anual a 10⁹ pet./mes)
- **Brechas** (ataques avanzados que evadieron)

Y un **registro de decisiones** en vivo: sospecha → ESCALA / BLOQUEA / sirve .

5. El Inspector paso a paso (/inspector)

Eliges una petición y ves **todo el proceso animado**, con los valores reales y el **código** de cada etapa. El backend (`trace_request`) devuelve la secuencia:

1. **Petición entrante** — IP, User-Agent, payload
2. **Edge extrae métricas** → `ip_frequency` , `ua_anomaly` , `payload_anomaly`
3. **Puntuación de sospecha** → $s = 0.15 \cdot \text{freq} + 0.15 \cdot \text{ua} + 0.70 \cdot \text{payload}$
4. **Decisión** (una de tres ramas):
 - bloqueo en Edge · vía rápida verde · **motor de Nash**
5. Si hubo Nash: **matrices + muestreo del "dado"** → escala o se queda

La barra de sospecha muestra `s` y el umbral `0.12` ; cada etapa incluye el fragmento de código real de `sge/` que la ejecuta.

6. Los 4 tipos de petición de ejemplo

Definidos en `_SAMPLES` (`demo/gateway.py`):

Botón	Clase real	Cómo se ve
legítimo	<code>LEGIT</code>	IP diversa, navegador real, payload corto y limpio
bot	<code>LIGHT_ATTACK</code>	UA <code>sqlmap/1.7</code> , IP repetida (alta frecuencia)
SQL injection	<code>ADVANCED_ATTACK</code>	UA de navegador, payload grande + <code>' ; DROP TABLE</code>
prompt injection	<code>ADVANCED_ATTACK</code>	UA de navegador, <i>"Ignore all previous instructions..."</i>

Los dos avanzados se **disfrazan** de navegador: solo los delata el **payload**.

7. Recorrido real — legítimo y bot

Legítimo → vía rápida verde (barato):

```
metricas: ip_freq=0.5  ua_anom=0.0  payload_anom=0.0  
sospecha: s = 0.012    (< umbral 0.12)  
decision: VIA RAPIDA VERDE → energia = 1 (vs 15 tradicional)
```

Bot ruidoso → bloqueo en el Edge (rate-limiting, sin gastar Serverless):

```
metricas: ip_freq=3.5  ua_anom=1.0  payload_anom=0.0  
sospecha: s = 0.216  
decision: BLOQUEO EN EL EDGE (UA anómalo + IP frecuente) → energia = 1
```

Ninguno de los dos ejecuta el motor de Nash: se resuelven al **costo mínimo**.

7. Recorrido real — ataque avanzado (SQLi)

Aquí **sí** se dispara el motor de Nash (tráfico sospechoso):

```
metricas: ip_freq=0.5  ua_anom=0.0  payload_anom=0.85
sospecha: s = 0.608

Matriz Defensor U_D = [[ 9.00, -94.35],      (filas D_V, D_A
                      [-3.00, -3.00]]      cols  A_L, A_M)

P(D_V Verde) = 0.202      P(D_A Escalar) = 0.798      tipo: mixed
dato aleatorio = 0.697 ≥ 0.202 → ESCALA a Serverless
```

Resultado: inspección profunda → amenaza neutralizada · energía = 16 (Edge 1 + Serverless 15).

El *prompt injection* es análogo: $s = 0.712$, $P(\text{escalar}) = 0.819$.

8. El "dado" — la naturaleza estocástica

La decisión final **no es fija**: se muestrea la estrategia mixta.

```
# sge/edge.py — la teoría se vuelve acción  
escalate = self._rng.random() ≥ nash.p_green
```

El botón "**Repetir 100 veces**" (/roll) lo demuestra sobre la MISMA petición:

```
prompt injection · P(escalar) = 0.819  
De 100 repeticiones: 82 ESCALARON · 18 se quedaron en Edge (BRECHA)
```

La proporción tiende a $P(\text{escalar})$ del Equilibrio de Nash. Para tráfico **benigno o bot** el camino es **determinista** (no hay dado).

9. Generador de carga y demo CLI

`demo/load_test.py` envía peticiones HTTP reales al gateway con la mezcla 80 % legit / 15 % bots / 5 % avanzados (mitad SQLi, mitad prompt injection):

```
python demo/load_test.py --n 2000 --url http://127.0.0.1:8000
```

Demo CLI (`main.py --demo-nash`): imprime el cálculo del Nash de una petición avanzada y luego el reporte de una simulación de N peticiones — misma lógica, sin navegador.

Ambos alimentan/usan el **mismo pipeline** `sge/`; cambia solo la interfaz.

10. Cómo leer los resultados

Salida real del panel tras **2000 peticiones** (`/simulate?n=2000`):

Métrica	Valor
Mezcla	1627 legit · 281 bots · 92 avanzados
Mitigación global	94.91 % (bots 100 %)
Mitigación avanzados	79.35 %
Escalados a Serverless	87 (4.35 % del tráfico)
Brechas (avanzado evadió)	19
Energía adaptativa vs. tradicional	3 805 J vs 30 010 J
Energía ahorrada	87.32 %
CO ₂ evitado (proyección)	~2.1 t/año

Solo **el ~4-5 % del tráfico** paga la inspección cara → casi toda la seguridad con una fracción de la energía. *(Los números varían por semilla; el patrón se mantiene.)*

En una frase

La demo enseña, **en vivo y con el código a la vista**, cómo SGE resuelve la mayoría del tráfico gratis en el Edge y **solo escala a la inspección cara el tráfico sospechoso** (guiado por el Equilibrio de Nash): **~95 % de mitigación con ~87 % menos energía.**

Web: `python demo/gateway.py` → <http://localhost:8000>

CLI: `python main.py --demo-nash`